

LECTURE 4

CONDITIONAL EXECUTION: ITERATION — DO-WHILE, WHILE, FOR

RECAP

- `char` DATATYPE
 - `char grade;`
 - `printf("%c",grade);`
 - `scanf("%c",&grade);`
 - `ctype.h` and library

RECAP

- LOGICAL OPERATORS
 - `==` `!=` `&&` `||` `!`

RECAP

- CONDITIONAL EXPRESSION
 - *logical_expression_1* ? *expression_2* : *expression_3*

RECAP

```
if(logical-expression){  
    // This block is executed if and only if the expression is TRUE  
    block-of-code  
}  
else{  
    // This block is executed if and only if the expression is FALSE  
    block-of-code  
}
```

RECAP

```
switch(expression){  
    case value_1:// This block is executed if and only if the expression==value_1  
one or more statements  
    break;  
    case value_2:// This block is executed if and only if the expression==value_2  
one or more statements  
    break;  
    case value_3:// This block is executed if and only if the expression==value_3  
one or more statements  
    break;  
}
```

RECAP

- WORKING OF `break;`
- `+=` `-=` `*=` `/=`
- `a++` `++a` `a--` `--a`

WHAT'S THE OUTPUT?

```
#include <stdio.h>
```

```
int main(void) {  
    int a=3,b=2;  
    a=a==b==0;  
    switch(1){  
        a=a+10;  
    }  
    printf("%d",a);  
    return 0;  
}
```

WHAT'S THE OUTPUT?

```
#include <stdio.h>

int main(void) {
    int a=3,b=2;
    a=a==b==0;           // a=1
    switch(1){
        a=a+10;          // not executed. Missing case 1:
    }
    printf("%d",a);      //1
    return 0;
}
```

CONDITIONAL EXECUTION: REPEATED ITERATIONS

DO-WHILE

MOTIVATION

- RECALL N-WAY FMAX
- REQUIRED REPEATEDLY EXECUTING FEW STATEMENTS IF A CONDITION WAS TRUE

DO-WHILE LOOP SYNTAX

DO-WHILE LOOP SYNTAX

```
do{  
    // This block is repeatedly executed as long as expression is TRUE  
    block-of-code  
}while(logical-expression);
```

FACTORIAL

```
#include <stdio.h>

int main(void){
    unsigned int number; printf("Enter number:\n"); scanf("%u",&number);
    long unsigned int factorial=1; //Note the datatypes employed
    printf("%u!=" ,number);

    printf("%lu.\n",factorial);
    return 0;}
```

FACTORIAL

```
#include <stdio.h>

int main(void){
    unsigned int number; printf("Enter number:\n"); scanf("%u",&number);
    long unsigned int factorial=1;
    printf("%u!=" ,number);

        do{                                //Conditionally repeated block
            factorial *= number--;
        }while(number >= 1);

    printf("%lu.\n",factorial);
    return 0;}
```

FACTORIAL

```
#include <stdio.h>

int main(void){
    unsigned int number; printf("Enter number:\n"); scanf("%u",&number);
    long unsigned int factorial=1;
    printf("%u!=",number);
    if(number>=2){                                     //0!,1! are 1, which is already initialized
        do{
            factorial *= number--;
        }while(number >= 1);
    }
    printf("%lu.\n",factorial);
    return 0;}
```

FACTORIAL

```
#include <limits.h>    // ULONG_MAX and other m/c specific constants are defined
#include <stdio.h>
int main(void){
    unsigned int number; printf("Enter number:\n"); scanf("%u",&number);
    long unsigned int factorial=1;
    printf("%u!=" ,number);
    if(number>=2){
        do{if(ULONG_MAX/number<factorial){printf("Overflow\n");return -1;}
            factorial *= number--;    //Check overflow
        }while(number >= 1);
    }
    printf("%lu.\n",factorial);
    return 0;}
```

WHAT'S THE OUTPUT?

```
#include <stdio.h>
int main(void) {
    char grade=0;
    do{
        printf("%c",grade++);
    }while(grade<=255);
    return 0;
}
```

RUNNING AVERAGE

RUNNING AVERAGE

```
#include <stdio.h>

int main(void){
    double sample, running_sum=0;
    int number_samples=0;
    do{
        printf("Enter next sample\'s value:\n"); scanf("%lf",&sample);++number_samples;
        running_sum += sample;
        printf("Running average=%lf.\n",running_sum/number_samples);

    }while(1); return 0;}
```

RUNNING AVERAGE

```
#include <stdio.h>

int main(void){
    double sample, running_sum=0;
    int number_samples=0; char proceed;
    do{
        printf("Enter next sample\'s value:\n"); scanf("%lf",&sample);++number_samples;
        running_sum += sample;
        printf("Running average=%lf.\n",running_sum/number_samples);
        printf("Enter \'y\' to continue to enter samples: ");scanf(" %c",&proceed);
        if(proceed!='y') break;    //Avoiding infinite loop
    }while(1); return 0;}
```

RUNNING AVERAGE

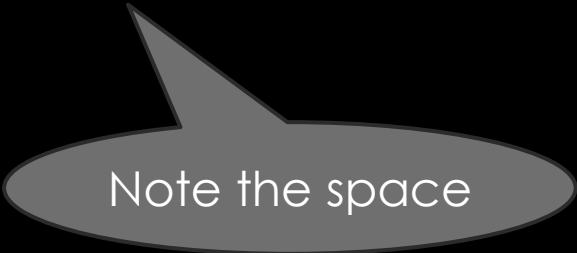
```
#include <stdio.h>

int main(void){
    double sample, running_sum=0;
    int number_samples=0; char proceed;
    do{
        printf("Enter next sample\'s value:\n"); scanf("%lf",&sample);++number_samples;
        running_sum += sample;
        printf("Running average=%lf.\n",running_sum/number_samples);
        printf("Enter \'y\' to continue to enter samples: ");scanf(" %c",&proceed);
        if(proceed!='y') break;    //return may not be appropriate if code follows
    }while(1); return 0;}
```

RUNNING AVERAGE

```
#include <stdio.h>

int main(void){
    double sample, running_sum=0;
    int number_samples=0; char proceed;
    do{
        printf("Enter next sample\'s value:\n"); scanf("%lf",&sample);++number_samples;
        running_sum += sample;
        printf("Running average=%lf.\n",running_sum/number_samples);
        printf("Enter \'y\' to continue to enter samples: ");scanf(" %c",&proceed);
        if(proceed!='y') break;
    }while(1); return 0;}
```



Note the space

RUNNING AVERAGE

```
#include <stdio.h>

int main(void){
    double sample, running_sum=0;
    int number_samples=0; char proceed;
    do{
        printf("Enter next sample\'s value:\n"); scanf("%lf",&sample);++number_samples;
        running_sum += sample;
        printf("Running average=%lf.\n",running_sum/number_samples);
        printf("Enter \'y\' to continue to enter samples: ");scanf(" %c",&proceed);
        if(proceed!='y') break;
    }while(1); return 0;}
```



```
scanf("%c%c",&proceed, &proceed);
```

break IS ALWAYS REPLACEABLE

```
#include <stdio.h>

int main(void){
    double sample, running_sum=0;
    int number_samples=0; char proceed;
    do{
        printf("Enter next sample\'s value:\n"); scanf("%lf",&sample);++number_samples;
        running_sum += sample;
        printf("Running average=%lf.\n",running_sum/number_samples);
        printf("Enter \'y\' to continue to enter samples: ");scanf(" %c",&proceed);
    }while(proceed=='y');
    return 0;}
```

WHILE LOOP SYNTAX

```
while(logical-expression){  
    // This block is repeatedly executed as long as expression is TRUE  
    block-of-code  
}
```

RUNNING AVERAGE

```
#include <stdio.h>

int main(void){
    double sample, running_sum=0;
    int number_samples=0; char proceed;
    while(1){
        printf("Enter next sample\'s value:\n"); scanf("%lf",&sample);++number_samples;
        running_sum += sample;
        printf("Running average=%lf.\n",running_sum/number_samples);
        printf("Enter \'y\' to continue to enter samples: ");scanf(" %c",&proceed);
        if(proceed!='y') break;
    } return 0;}
```

RUNNING AVERAGE (NO break)

```
#include <stdio.h>

int main(void){
    double sample, running_sum=0;
    int number_samples=0; char proceed='y';
    while(proceed=='y'){
        printf("Enter next sample\'s value:\n"); scanf("%lf",&sample);++number_samples;
        running_sum += sample;
        printf("Running average=%lf.\n",running_sum/number_samples);
        printf("Enter \'y\' to continue to enter samples: ");scanf(" %c",&proceed);

    } return 0;}
```

FOR SKIPPING TYPOS

continue FOR SKIPPING TYPOS

```
#include <stdio.h>

int main(void){
    double sample, running_sum=0;
    int number_samples=0; char proceed='y';
    while(proceed=='y'){
        printf("Enter next sample\'s value:\n"); scanf("%lf",&sample);
        if((sample>1)|| (sample<0)) continue;
        running_sum += sample;
        printf("Running average=%lf.\n",running_sum/++number_samples);
        printf("Enter \'y\' to continue to enter samples: ");scanf(" %c",&proceed);
    }
    return 0;}
```

FACTORIAL

$$n! \equiv \prod_{i=1}^n i$$

$$n! \equiv e^{\sum_{i=1}^n \ln(i)}$$

FOR LOOP SYNTAX

```
for(init_expr; logical_expr; increment_expr){  
    // This block is repeatedly executed as long as expression is TRUE  
    block-of-code  
}
```

FOR LOOP SYNTAX

```
for(int i=1;i<=n;i++){  
    // This block is repeatedly executed as  
    long as expression is TRUE  
    block-of-code  
}
```

Fortran

```
DO 10 I = 1, 10
```

```
...
```

Algol

```
for i := 1 step 1 until 10 do  
    statement;
```

FACTORIAL

```
#include <stdio.h>

int main(void){
    unsigned int number; printf("Enter number:\n"); scanf("%u",&number);
    long unsigned int factorial=1;

    printf("%u!=%lu.\n",number,factorial);
    return 0;
}
```

FACTORIAL

```
#include <stdio.h>

int main(void){
    unsigned int number; printf("Enter number:\n"); scanf("%u",&number);
    long unsigned int factorial=1;
    for(unsigned int i=2;i<=number;i++){ // index i is defined inside for block
        factorial *= i;
    }
    printf("%u!=%lu.\n",number,factorial);
    return 0;
}
```

WHAT'S THE OUTPUT?

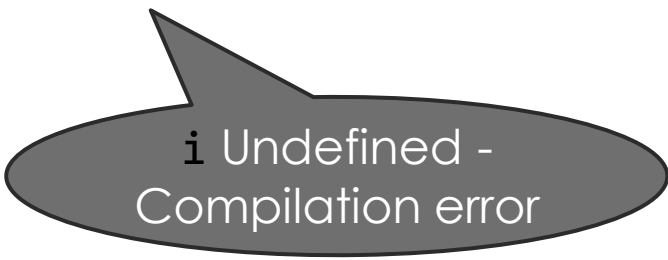
```
#include <stdio.h>

int main(void){
    unsigned int number; printf("Enter number:\n"); scanf("%u",&number);
    long unsigned int factorial=1;
    for(unsigned int i=2;i<=number;i++){
        factorial *= i;
    }
    printf("%u!=%lu.\n",number,factorial);printf("%u\n",i);
    return 0;
}
```

WHAT'S THE OUTPUT?

```
#include <stdio.h>

int main(void){
    unsigned int number; printf("Enter number:\n"); scanf("%u",&number);
    long unsigned int factorial=1;
    for(unsigned int i=2;i<=number;i++){
        factorial *= i;
    }
    printf("%u!=%lu.\n",number,factorial);printf("%u\n",i);
    return 0;
}
```



i Undefined -
Compilation error

SCOPE OF VARIABLES

- VARIABLES'S EXISTENCE IS LIMITED TO THE BLOCK THEY ARE DEFINED IN
 - EXISTENCE IS INHERITED BY CHILD BLOCKS
 - FOR E.G., VARIABLES IN MAIN WILL BE AVAILABLE IN ALL SUBSEQUENT IF/SWITCH/WHILE/FOR BLOCKS
- VARIABLES WITH THE SAME NAME MAY CO-EXIST INSIDE DIFFERENT BLOCKS
 - IF THE BLOCKS ARE NESTED, LOCAL VARIABLE IS ACTIVE

WHAT'S THE OUTPUT?

```
#include <stdio.h>

int main(void){
    int i=3;
    if(1){
        int i=4;printf("inside if i=%d\n",i);
    }
    for(int i=-5;i<=-3;i+=2) printf("loop i=%d\n",i);
    // {} may be skipped for single-statement blocks
    printf("main i=%d\n",i);
    return 0;
}
```

WHAT'S THE OUTPUT?

```
#include <stdio.h>

int main(void){
    int i=3;
    if(1){
        int i=4;printf("inside if i=%d\n",i);        // inside if i=4
    }
    for(int i=-5;i<=-3;i+=2) printf("loop i=%d\n",i); // loop i=-5 loop i=-3

    printf("main i=%d\n",i);                          // main i=3
    return 0;
}
```

MEMORY ALLOCATION

enter main



↓ enter if-block



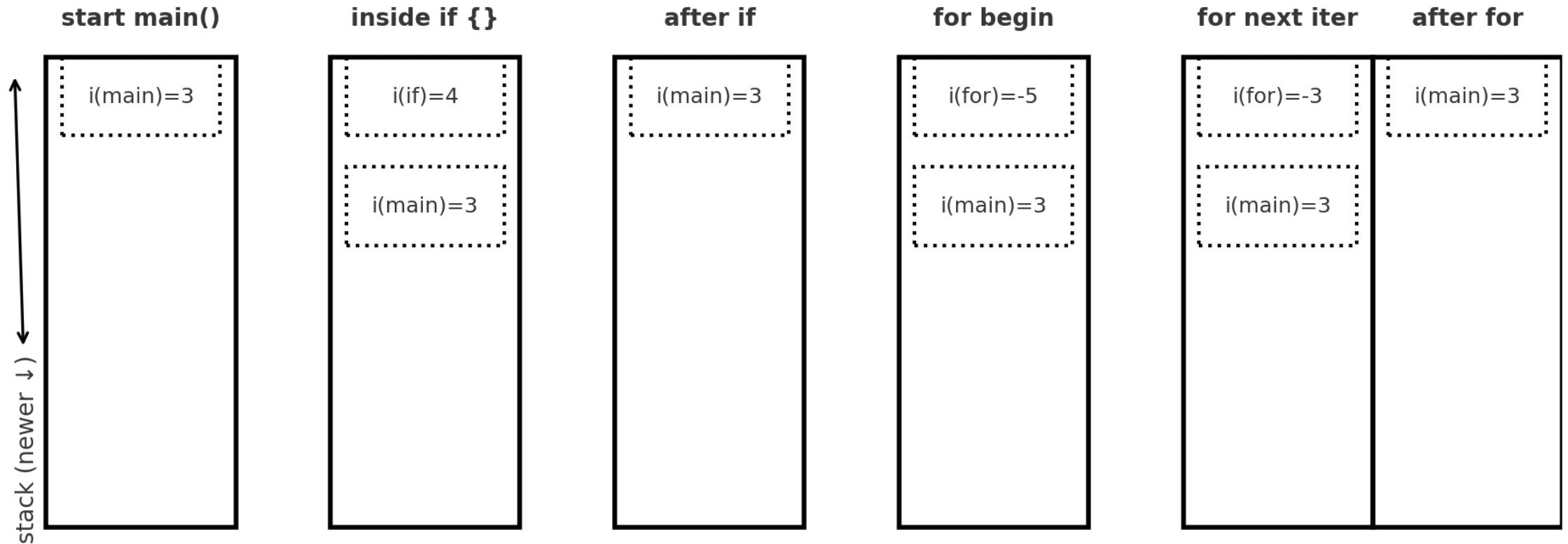
↓ leave if-block → (B) destroyed; (A) visible again
enter for-statement



leave for-statement → (C) destroyed; (A) visible again
print main i → uses (A)
exit main → (A) destroyed

MEMORY ALLOCATION

Memory snapshots (stack) for three different 'i' variables



MEMORY ALLOCATION

```
#include <stdio.h>
int main(void){
    int i = 3;
    printf("main i=%d @%p\n", i, (void*)&i);

    if (1){
        int i = 4;
        printf("if i=%d @%p\n", i, (void*)&i);
    }

    for (int i = -5; i <= -3; i += 2){
        if (i == -5) printf("for i first iter @%p\n", (void*)&i);
        printf("loop i=%d\n", i);
    }

    printf("main i=%d @%p\n", i, (void*)&i);
    return 0;
}
```

WHAT'S THE OUTPUT?

```
#include <stdio.h>

int main(void){
    int i=3;
    if(1){
        i=4;printf("inside if i=%d\n",i);
    }
    for(i=-5;i<=-3;i+=2) printf("loop i=%d\n",i);
    printf("main i=%d\n",i);
    return 0;
}
```

WHAT'S THE OUTPUT?

```
#include <stdio.h>

int main(void){
    int i=3;
    if(1){
        i=4;printf("inside if i=%d\n",i);        // inside if i=4
    }
    for(i=-5;i<=-3;i+=2) printf("loop i=%d\n",i); // loop i=-5 loop i=-3
    printf("main i=%d\n",i);                     // main i=-1
    return 0;
}
```

FACTORIAL

$$n! \equiv \prod_{i=1}^n i$$

$$n! \equiv e^{\sum_{i=1}^n \ln(i)}$$

FACTORIAL

```
#include <stdio.h>
#include <math.h>
int main(void){
    unsigned int number; printf("Enter number:\n"); scanf("%u",&number);
    double factorial=0;
    for(unsigned int i=2;i<=number;i++){
        factorial += log(i);
    }
    printf("%u!=%e.\n",number,exp(factorial));
    return 0;
}
```

FACTORIAL

```
#include <stdio.h>
#include <math.h>
int main(void){
    unsigned int number; printf("Enter number:\n"); scanf("%u",&number);
    double factorial=0;
    for(unsigned int i=2;i<=number;i++){
        factorial += log(i);
    }
    printf("%u!=%e.\n",number,exp(factorial));
    return 0;
}
```



Exp format

WHAT'S THE OUTPUT?

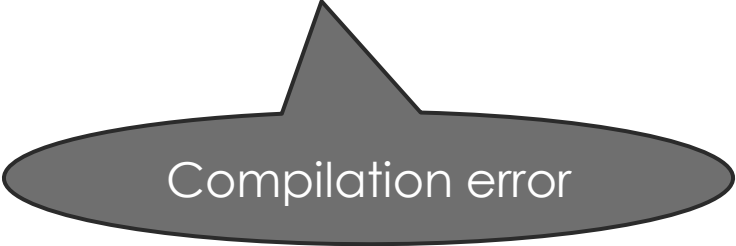
```
#include <stdio.h>

int main(void){
    int i=3;
    switch(i){
        case 3: int j=4;printf("inside case 3 j=%d\n",j);break;
        case 1: int j=5;printf("inside case 1 j=%d\n",j);break;
    }
    return 0;
}
```

WHAT'S THE OUTPUT?

```
#include <stdio.h>

int main(void){
    int i=3;
    switch(i){
        case 3: int j=4;printf("inside case 3 j=%d\n",j);break;
        case 1: int j=5;printf("inside case 1 j=%d\n",j);break;
    }
    return 0;
}
```



Compilation error

PRIMALITY TEST

PRIMALITY TEST

```
#include <stdio.h>
```

```
int main(void){
```

```
    unsigned int number; printf("Enter number:\n"); scanf("%u",&number);
```

```
    return 0;}
```

PRIMALITY TEST

```
#include <stdio.h>
```

```
int main(void){  
    unsigned int number; printf("Enter number:\n"); scanf("%u",&number);  
    for(unsigned int i=2;i<=    number ;i++){  
        if(number%i==0){  
            printf("%u is NOT a prime. It is divisible by %d.\n",number,i);  
            return 0;  
        }  
    }  
    printf("%u is a prime.\n",number);  
    return 0;}
```

PRIMALITY TEST

```
#include <stdio.h>
#include <math.h>
int main(void){
    unsigned int number; printf("Enter number:\n"); scanf("%u",&number);
    for(unsigned int i=2;i<=sqrt(number);i++){
        if(number%i==0){
            printf("%u is NOT a prime. It is divisible by %d.\n",number,i);
            return 0;
        }
    }
    printf("%u is a prime.\n",number);
    return 0;}
```

PRIMALITY TEST

```
#include <stdio.h>
#include <math.h>
int main(void){
    unsigned int number; printf("Enter number:\n"); scanf("%u",&number);
    for(unsigned int i=2;i<=sqrt(number);i++){
        if(number%i==0){
            printf("%u is NOT a prime. It is divisible by %d.\n",number,i);
            return 0;
        }
    }
    printf("%u is a prime.\n",number);
    return 0;}
```

WHAT'S THE OUTPUT?

```
#include <stdio.h>
int main(void) {
    while(printf("Hello! "));
    return 0;
}
```

PRINT A DIAMOND SHAPE

[illegible]

PRINT A DIAMOND SHAPE

- LET R DENOTE THE NUMBER OF ROWS (ODD)
- NO. STARS IN r^{th} ROW: $2 \min(r, R - r + 1) - 1$

- SPACES BEFORE STARS: $\frac{(R - (2 \min(r, R - r + 1) - 1))}{2}$

- SPACES AFTER STARS: $\frac{(R - (2 \min(r, R - r + 1) - 1))}{2}$

PRINT A DIAMOND SHAPE

- LET R DENOTE THE NUMBER OF ROWS (ODD)
- NO. STARS IN r^{th} ROW: $2 \min(r, R - r + 1) - 1$
 - SPACES BEFORE STARS: $\frac{(R - (2 \min(r, R - r + 1) - 1))}{2}$
 - SPACES AFTER STARS: $\frac{(R - (2 \min(r, R - r + 1) - 1))}{2}$

PRINT A DIAMOND SHAPE

```
#include <stdio.h>

int main(void){
    unsigned int totalRows; printf("Enter an odd number:\n"); scanf("%u",&totalRows);
    unsigned int no_spaces, no_stars;
    for(unsigned int r=1;r<=totalRows;r++){        // iterate over each row

    }

    return 0;}
```

PRINT A DIAMOND SHAPE

```
#include <stdio.h>

int main(void){
    unsigned int totalRows; printf("Enter an odd number:\n"); scanf("%u",&totalRows);
    unsigned int no_spaces, no_stars;
    for(unsigned int r=1;r<=totalRows;r++){
        no_stars=2*((r<=(totalRows-r+1)) ? r : (totalRows-r+1))-1;
        no_spaces=(totalRows-no_stars)/2;

    }
    return 0;}
```

PRINT A DIAMOND SHAPE

```
#include <stdio.h>

int main(void){
    unsigned int totalRows; printf("Enter an odd number:\n"); scanf("%u",&totalRows);
    unsigned int no_spaces, no_stars;
    for(unsigned int r=1;r<=totalRows;r++){
        no_stars=2*((r<=(totalRows-r+1)) ? r : (totalRows-r+1))-1;
        no_spaces=(totalRows-no_stars)/2;
        for(unsigned int i=1;i<=no_spaces;i++) printf(" ");
        for(unsigned int i=1;i<=no_stars;i++) printf("*");
        for(unsigned int i=1;i<=no_spaces;i++) printf(" "); printf("\n");
    }
    return 0;}
```

break AND continue

- BOTH CAN BE USED IN DO, WHILE AND FOR
 - `break` CAN BE USED IN SWITCH TOO
- `continue` IN FOR WILL NOT SKIP THE INDEX INCREMENT/DECREMENT